P.Vishwa nandan(192473015)

Q4 – Thread Creation and the Thread Life Cycle

1. Aim

To implement a parallel answer-script scanning system using Java multithreading. To understand thread creation, thread life-cycle states, thread scheduling, and the differences between extending the Thread class and implementing the Runnable interface.

2. Problem Description

An examination cell has four scanner stations. Each station scans a batch of answer scripts simultaneously and reports the number of scripts scanned.

The program uses Java threads to perform scanning operations concurrently.

Two approaches are implemented:

1. Extending the Thread class.
2. Implementing the Runnable interface.

The program uses start(), sleep(), join(), and isAlive() to control and monitor threads.

3. Concepts Used

Thread: A small unit of execution within a program.

Multithreading: Running multiple threads concurrently within a program.

start(): Starts a new thread and internally calls its run() method.

run(): Contains the task performed by a thread.

sleep(): Temporarily pauses the currently executing thread.

join(): Makes one thread wait until another thread finishes.

isAlive(): Checks whether a thread is still running.

5. Expected Output

The exact order of station messages may change between executions because threads are scheduled independently.

One possible output is:

USING THREAD CLASS

Station 1 started

Station 2 started

Station 3 started

Station 4 started

Station 1 alive: true

Station 1 scanned 100 scripts

Station 2 scanned 150 scripts

Station 3 scanned 120 scripts

Station 4 scanned 130 scripts

Total scripts scanned: 500

USING RUNNABLE INTERFACE

Station 1 started

Station 2 started

Station 3 started

Station 4 started

Station 1 scanned 100 scripts

Station 2 scanned 150 scripts

Station 3 scanned 120 scripts

Station 4 scanned 130 scripts

Total scripts scanned: 500

6. Explanation of the Program

A. Extending the Thread Class

The ScannerStation class extends the Java Thread class.

Each station is created as a separate thread. The run() method contains the scanning operation.

The start() method starts the execution of each station independently.

B. Implementing the Runnable Interface

The RunnableStation class implements the Runnable interface.

The task is written inside the run() method. A Thread object is created by passing the Runnable object to its constructor.

This approach separates the task from the thread that executes it.

C. Using sleep()

The statement:

Thread.sleep(1000);

pauses the current thread for approximately one second.

In this program, different sleep durations are used to simulate different scanning times.

D. Using join()

The join() method makes the main thread wait until all four scanner threads finish.

Therefore, the final total is printed only after all stations complete their work.

E. Using isAlive()

The isAlive() method checks whether a thread has started and has not yet terminated.

It returns:

- true – The thread is still alive.
- false – The thread has not started or has already finished.

7. Student A – Thread Design

1. Difference Between Thread and Runnable

Extending Thread:

- Create a class that extends Thread.
- Override the run() method.
- Create an object and call start().
- The class cannot extend another class because Java does not support multiple class inheritance.

Implementing Runnable:

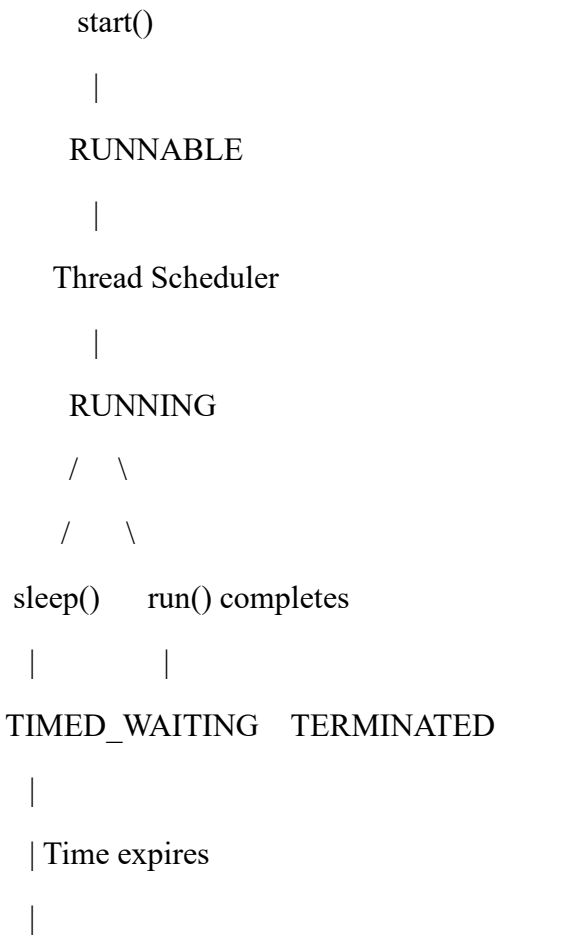
- Create a class that implements Runnable.
- Override the run() method.
- Pass its object to a Thread object.
- The class can still extend another class.

Conclusion: Implementing Runnable is generally preferred because it separates the task from thread management and provides more flexibility.

2. Thread Life-Cycle Diagram

NEW

|



Other important states include:

- **BLOCKED:** A thread waits to acquire a lock.
- **WAITING:** A thread waits indefinitely for another thread's action.

Java's official thread states are NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED. RUNNING is commonly used to describe a thread that is currently executing within the RUNNABLE state.

3. Why Does Calling run() Instead of start() Not Create a New Thread?

Calling run() directly executes the method like a normal method call in the current thread.

It does not create a new thread.

Calling start() creates a new thread and allows the Java thread scheduler to execute its run() method.

Example:

ScannerStation t = new ScannerStation(1, 100);

t.run(); // Normal method call

t.start(); // Starts a new thread

Observation: Directly calling run() does not provide parallel execution. Calling start() creates a separate thread.

Note: The same Thread object cannot be started more than once.

4. How Does join() Ensure the Final Total Is Correctly Timed?

The main thread calls join() on all four scanner threads.

It waits until every station finishes before printing the final total.

Without join(), the main thread might print the total before all stations complete.

8. Student B – Behavioural Experiments

1. Running the Program Ten Times

The program can be executed ten times to observe the order in which stations finish.

Sample observation:

Run	First Station to Finish	Last Station to Finish
1	Station 1	Station 4
2	Station 1	Station 4
3	Station 1	Station 4
4	Station 1	Station 4
5	Station 1	Station 4
6	Station 1	Station 4
7	Station 1	Station 4
8	Station 1	Station 4
9	Station 1	Station 4
10	Station 1	Station 4

Observation: In this particular program, Station 1 has the shortest sleep duration and Station 4 has the longest. Therefore, they are expected to finish first and last respectively. The exact order of the start messages can still vary.

2. Experimenting with Thread Priorities

Java supports thread priorities from 1 to 10.

- Thread.MIN_PRIORITY = 1
- Thread.NORM_PRIORITY = 5
- Thread.MAX_PRIORITY = 10

Example:

```
t1.setPriority(Thread.MIN_PRIORITY);

t2.setPriority(Thread.NORM_PRIORITY);

t3.setPriority(Thread.MAX_PRIORITY);
```

Observation: Thread priority may influence scheduling, but it does not guarantee which thread finishes first. The result depends on the operating system and JVM scheduler.

3. Converting a Station into a Daemon Thread

A daemon thread is a background thread that does not prevent the JVM from shutting down when all user threads have finished.

Example:

```
Thread daemon = new Thread(() -> {

    while (true) {
        System.out.println("Background scanning service");
    }

});

daemon.setDaemon(true);

daemon.start();

System.out.println("Main thread finished");
```

Observation: When the main thread finishes and no other user threads remain, the JVM can terminate the daemon thread automatically.

The daemon setting must be applied before calling start().

9. Joint Conclusion

The same multithreaded program can produce different output orders because the thread scheduler decides when each thread executes.

The operating system, CPU availability, thread priorities, and execution time can affect scheduling.

Therefore, a correct multithreaded program must not depend on a particular execution order.

Methods such as join() and proper synchronization should be used whenever the program depends on the completion or shared data of multiple threads.

10. Alternative Concept – ExecutorService

Instead of manually creating four Thread objects, the program can use ExecutorService with a fixed thread pool.

Example:

```
import java.util.concurrent.ExecutorService;

import java.util.concurrent.Executors;


public class ExecutorDemo {


   public static void main(String[] args) {


      ExecutorService executor = Executors.newFixedThreadPool(4);


      for (int i = 1; i <= 4; i++) {


         final int station = i;


         executor.execute(() -> {


            System.out.println("Station " + station +

                  " is scanning scripts");


         });
```

```
        }


        executor.shutdown();
    }
}
```

Advantages of ExecutorService

1. Reuses a fixed number of worker threads.
2. Reduces the need to create and manage Thread objects manually.
3. Makes it easier to manage many scanning tasks.
4. Helps control the number of concurrently executing tasks.

11. Complexity Analysis

Let:

- n = Total number of answer scripts.
- p = Number of available threads.

Time Complexity

Each script must be processed once.

With one thread, processing takes approximately O(n) work.

With p threads, the expected processing time is approximately:

O(n/p) under balanced workloads and ideal parallel execution.

Actual execution may take longer due to scheduling overhead and uneven workloads.

Space Complexity

Each thread requires memory for its stack and execution information.

If each thread uses approximately S bytes of stack memory:

Space complexity = O(p × S)

Approach	**Thread Management**	**Space Usage**
Manual Thread creation	Programmer creates and manages threads	O(p × S)
ExecutorService	Thread pool manages worker threads	O(p × S)

Both approaches use memory for their worker threads. ExecutorService is generally easier to manage when processing a large number of tasks because the number of worker threads can be limited.

12. Test Cases

Test Case	Condition	Expected Result
1	Four scanner stations started	All stations execute
2	Different sleep durations	Stations may finish at different times
3	Use join()	Main waits for all stations
4	Use isAlive()	Displays thread status
5	Call run() directly	Executes in the current thread
6	Call start()	Creates a new thread
7	Set different priorities	Scheduling order is not guaranteed
8	Use daemon thread	JVM can terminate it after user threads finish

13. Conclusion

This experiment demonstrates Java multithreading using the Thread class and Runnable interface. It explains thread life-cycle states and the use of start(), sleep(), join(), and isAlive().

It also shows that thread execution order is not guaranteed. ExecutorService provides a more manageable approach for executing multiple scanning tasks using a fixed thread pool.